

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

C PROGRAMMING ASSIGNMENT 1

Design, Implement and Validate a Reliable Billing Program for Rural Electricity Co-operatives

Submitted by: Janushiya P A

Register Number: 1924260368

TASK 1 – Understand the Problem and Identify the Data

1. Aim

To analyse the electricity-billing problem and identify the inputs, outputs, constants, variables, rules, and assumptions required to develop a correct and reliable C program.

2. Problem Analysis

Grama Jyothi Rural Electricity Co-operative needs a C program to calculate household electricity bills. The program uses meter readings to calculate electricity consumption, applies the correct tariff, adds the fixed charge, and displays the total bill accurately to the paisa.

3. Inputs and Their Importance

- **Previous Reading:** The meter reading at the beginning of the billing period. It is required to calculate electricity consumption.
- **Current Reading:** The latest meter reading. Comparing it with the previous reading determines the units consumed.
- **Household Number:** Identifies each consumer and associates the bill with the correct household.
- **Household Category:** Determines the applicable tariff if different categories have different billing rates.

4. Outputs and Their Importance

- **Units Consumed:** The difference between the current and previous readings, adjusted for a valid meter rollover.
- **Energy Charge:** The amount calculated according to the electricity tariff slabs.
- **Fixed Charge:** The fixed amount charged for the billing period.
- **Total Bill:** The sum of the energy charge and fixed charge, including any other required charges.

5. Constants and Variables

Constants: Tariff rates, slab limits, fixed charge, and meter maximum value remain unchanged during a billing calculation. Named constants make the program easier to understand and maintain.

Variables: Meter readings, household number, category, units consumed, energy charge, and total bill may change for each household. Therefore, they must be stored in variables.

6. Negative, Fractional, and Large Values

Valid meter readings and units consumed cannot be negative. A smaller current reading may indicate meter rollover or an incorrect reading, so it must be checked.

Monetary calculations must be accurate to the paisa. Therefore, storing money as integer paise is preferable to using floating-point rupees. Large household counts and district-wide totals require data types with sufficient storage capacity.

7. Rules for a Correct Bill

1. Validate all inputs before calculation.
2. Calculate consumption correctly, including confirmed meter rollover.
3. Apply the correct tariff and household category.
4. Ensure the total equals the sum of all bill components.
5. Ensure the bill is not negative under the stated billing rules.
6. Display the amount accurately to two decimal places.

8. Assumptions

Each household has a unique number, and the co-operative provides the applicable tariff rates, slab limits, fixed charges, and category rules. If a 16-bit unsigned meter register is used, its values range from 0 to 65535. A decrease in reading is treated as rollover only when the meter's rollover behaviour is confirmed.

9. Alternative Concept: Text vs. Integer Readings

A meter reading can be stored as a character string, such as "01530", instead of an integer.

Text preserves leading zeros, but the program must validate and convert it before performing calculations. For a reading containing d digits, conversion and validation require **O(d) time** and **O(d) storage**. A fixed-size integer requires **O(1) storage**, and comparison and arithmetic take constant time for a fixed-width type.

Therefore, integers are more suitable for billing calculations, while strings are useful for preserving the original reading format.

10. Conclusion / Result

The analysis identifies the data and rules required for accurate electricity billing. Using suitable data types, named constants, input validation, correct rollover handling, and integer paise helps produce reliable bills for individual households and large numbers of consumers.

TASK 2 – CHOOSE THE DATA TYPES

1. Aim

To select suitable C data types for an electricity billing system and calculate meter consumption correctly, including meter register rollover.

2. Data Type Table

Data item	Data type	Size (typical)	Range / purpose
Meter reading	unsigned short	2 bytes	0 to 65,535
Units consumed	unsigned int	4 bytes	Non-negative consumption
Consumer count	unsigned int	4 bytes	Stores consumer numbers
Money in paise	long long	8 bytes	Exact integer money storage
Decimal calculation	double	8 bytes	Approximate floating-point values

Sizes and ranges depend on the compiler and system. sizeof checks the size, limits.h provides integer limits, and float.h provides floating-point limits.

Why integer paise? Floating-point values cannot represent all decimal amounts exactly. For 4,000 bills of ₹0.10, float accumulation can give approximately ₹400.015 instead of ₹400.00. Storing 10 paise per bill gives exactly 40,000 paise = ₹400.00, provided no integer overflow occurs.

3. Algorithm

1. Start.
2. Read the previous and current meter readings.
3. Subtract the previous reading from the current reading using unsigned 16-bit arithmetic.
4. Print the consumption.
5. Stop.

4. Code

main.c Run ▶

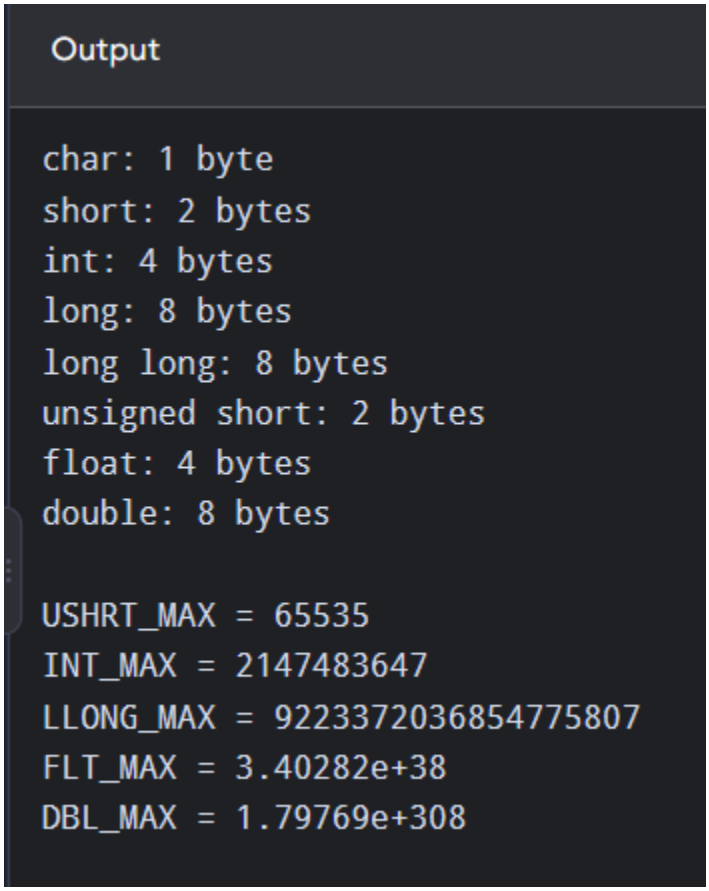
```c
#include <stdio.h>
#include <limits.h>
#include <float.h>

int main(void)
{
    printf("char: %zu byte\n", sizeof(char));
    printf("short: %zu bytes\n", sizeof(short));
    printf("int: %zu bytes\n", sizeof(int));
    printf("long: %zu bytes\n", sizeof(long));
    printf("long long: %zu bytes\n", sizeof(long long));
    printf("unsigned short: %zu bytes\n", sizeof(unsigned short));
    printf("float: %zu bytes\n", sizeof(float));
    printf("double: %zu bytes\n", sizeof(double));

    printf("\nUSHRT_MAX = %u\n", (unsigned)USHRT_MAX);
    printf("INT_MAX = %d\n", INT_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    printf("FLT_MAX = %g\n", (double)FLT_MAX);
    printf("DBL_MAX = %g\n", DBL_MAX);

    return 0;
}
```

6. Output



7. Explanation

A 16-bit unsigned register stores values from 0 to 65,535. After 65,535, it rolls over to 0.

For the second test: Consumption = (65,536 − 65,400) + 120 = **256 units**.

The result is positive because unsigned 16-bit arithmetic wraps modulo 65,536.

8. Complexity Analysis

- Time per addition: O(1) for both integer paisa and double.
- Total time for n bills: O(n).
- Extra space for a running total: O(1).
- Integer paise avoid rounding errors in addition, while double may accumulate small floating-point errors. Both typically occupy 8 bytes for long long and double, respectively.

If money is stored as double rupees, rounding to two decimal places is normally applied when converting the final total to a displayed bill amount.

9. Result

Suitable C data types were selected, and the program successfully calculated normal meter consumption and rollover consumption. Integer paisas were identified as the preferred method for accurate money calculations.

TASK 3 – DEFINE THE CONSTANTS

1. Aim

To define electricity tariff rates and billing limits as constants so that tariff revisions can be made easily in one place.

2. Algorithm

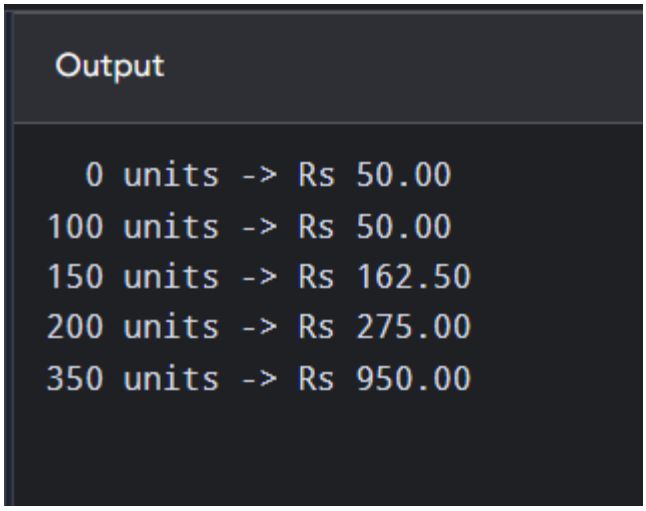
1. Start.
2. Define the tariff slab limits, rates, and fixed charge using #define.
3. Read the electricity units consumed.
4. Initialize the bill with the fixed charge of ₹50.
5. Calculate the charge for units above 200 using RATE3_PAISE.
6. Calculate the charge for units between 101 and 200 using RATE2_PAISE.
7. Calculate the charge for the first 100 units using RATE1_PAISE.
8. Add the slab charges to the fixed charge.
9. Convert the total paise into rupees and paise and display the bill.
10. Repeat for all test inputs.
11. Stop.

3. Code

```c
#include <stdio.h>
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000L
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > SLAB2_UNITS)
        paise += (long)(units - SLAB2_UNITS) * RATE3_PAISE;
    if (units > SLAB1_UNITS) {
        int slab2 = units < SLAB2_UNITS ?
                    units - SLAB1_UNITS :
                    SLAB2_UNITS - SLAB1_UNITS;
        paise += (long)slab2 * RATE2_PAISE;
    }
    if (units > 0 && units <= SLAB1_UNITS)
        paise += (long)units * RATE1_PAISE;
    return paise;
}
int main(void)
{
    int tests[] = {0, 100, 150, 200, 350};
    int n = sizeof tests / sizeof tests[0];

    for (int i = 0; i < n; i++) {
        long p = compute_bill(tests[i]);
        printf("%3d units -> Rs %ld.%02ld\n",
               tests[i], p / 100, p % 100);
    }
    return 0;
}
```

4. Output



5. Result

The tariff rates and fixed charge were defined as constants, and the electricity bill was calculated according to the slab system. Updating a single tariff constant allows the revised rate to be applied consistently throughout the program.

TASK 4 – USE MACROS SAFELY

1. Aim

To define safe C macros for repeated calculations and understand macro expansion, operator precedence, repeated argument evaluation, and the advantages of inline functions.

2. Algorithm

1. Start.
2. Define macros for converting rupees to paise, squaring a value, and finding the maximum of two values.
3. Use parentheses to ensure correct operator precedence.
4. Test the macros with sample values.
5. Display the results.
6. Compare macros with static inline functions.
7. Stop.

3. Code

```c
#include <stdio.h>
#define RUPEES_TO_PAISE(r) ((r) * 100)
#define SQUARE(x) ((x) * (x))
#define MAX(a, b) (((a) > (b)) ? (a) : (b))
static inline int square(int x)
{
    return x * x;
}
static inline int maximum(int a, int b)
{
    return (a > b) ? a : b;
}
int main(void)
{
    int a = 4, b = 7;
    printf("Rupees to paise: %d\n", RUPEES_TO_PAISE(10));
    printf("Square of (a+1): %d\n", SQUARE(a + 1));
    printf("Maximum: %d\n", MAX(a, b));
    printf("Inline square: %d\n", square(a + 1));
    printf("Inline maximum: %d\n", maximum(a, b));
    return 0;
}
```

4. Output

Output

```
Rupees to paise: 1000
Square of (a+1): 25
Maximum: 7
Inline square: 25
Inline maximum: 7
```

5. Explanation of Macro Pitfalls

a) Pre-processor expansion

The pre-processor replaces macro calls with their expanded expressions before compilation. The command below displays the expanded source code:

gcc -E task4.c

For example, SQUARE(a + 1) expands to:

((a + 1) * (a + 1))

b) Missing parentheses

If the macro is written as #define SQUARE(x) x*x, then SQUARE(a+1) expands to a+1*a+1. For a = 4, the result is 9 instead of 25 because multiplication has higher precedence than addition. Parentheses around the parameter and the complete expression fix this problem.

c) Repeated evaluation in MAX

The macro MAX (a, b) may evaluate an argument more than once. For example, MAX(i++, j) can increment i twice if i++ is selected by both the comparison and the final expression. Avoid side effects in macro arguments.

d) When a function is better

A function is better when arguments have side effects, type checking is important, or the calculation is complex. A static inline function provides normal parameter evaluation and clearer debugging.

6. Alternative Concept and Complexity Analysis

- **Time:** Both macros and inline functions generally perform a constant number of operations, O(1), for these calculations. A normal function call may have small call overhead, while an inline function may be optimized to avoid it.
- **Code size:** A macro is expanded at every use, so using it (k) times may duplicate its expression (k) times. An inline function may also be expanded by the compiler, but the compiler decides whether to inline it or retain a function call.
- **Space:** Macros have no independent runtime storage. Inline functions may generate a function body or duplicated instructions, depending on compiler optimization.

7. Result

Safe macros were implemented using parentheses to avoid operator-precedence errors. The program demonstrated rupees-to-paise conversion, squaring, and finding the maximum value.

The use of gcc -E shows how macros are expanded before compilation, and inline functions provide a safer alternative when repeated argument evaluation is a concern.

TASK 5 – ORGANISE THE PROGRAM WITH HEADER FILES AND CONDITIONAL COMPILATION

1. Aim

To organise constants and function prototypes in a header file and use conditional compilation to display debugging information only when DEBUG is enabled.

2. Algorithm

1. Start.
2. Create a header file containing constants, function prototypes, and include guards.
3. Define the billing function in a separate C source file.
4. Use #ifdef DEBUG to print intermediate values when debugging is enabled.
5. Compile and execute the program with and without -DDEBUG.
6. Display the output and stop.

3. Program Files

The program consists of three files:

- `billing.h` – contains constants, the function prototype, and include guards.
- `billing.c` – contains the billing function and debugging statements.
- `main.c` – contains the main function and displays the bill.

4. Explanation

Header files: Store constants, declarations, and function prototypes.

Include guards: `#ifndef`, `#define`, and `#endif` prevent a header file from being included repeatedly in the same translation unit.

Conditional compilation: `#ifdef DEBUG` includes debugging statements only when `DEBUG` is defined using `-DDEBUG`.

Difference between include statements:

- `#include <stdio.h>` is used for standard library headers.
- `#include "billing.h"` is used for a user-created header file and normally
- Searches the local directory first.

Code

```c
#include <stdio.h>
#define FIXED_CHARGE_PAISE 5000L
#define RATE2_PAISE 225
#define RATE3_PAISE 450
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;
    if (units > 200)
        paise += (long)(units - 200) * RATE3_PAISE;
    if (units > 100) {
        int slab2 = units < 200 ? units - 100 : 100;
        paise += (long)slab2 * RATE2_PAISE;
    }
#ifdef DEBUG
    printf("DEBUG: units = %d, bill = %ld paise\n",
           units, paise);
#endif
    return paise;
}
int main(void)
{
    int units = 150;
    long bill = compute_bill(units);
    printf("Units = %d\n", units);
    printf("Bill = Rs %ld.%02ld\n",
           bill / 100, bill % 100);
    return 0;
}
```

5. Output without Debugging

```
Output

Units = 150
Bill = Rs 162.50
```

6. Output with Debugging

```
Output

DEBUG: units = 150, bill = 16250 paise
Units = 150
Bill = Rs 162.50
```

7. Complexity Analysis

The billing calculation takes O(1) time per call and O(1) auxiliary space.

With `#ifdef DEBUG`, debugging statements are excluded from the program when debugging is disabled, avoiding runtime checks. With `if (debug)`, the program performs a runtime condition check, and the debug code remains compiled into the executable. The additional test takes O(1) time per call.

8. Result

The C program was organised using a header file, include guards, and conditional compilation. Debugging output is displayed when compiled with `-DDEBUG` and omitted when debugging is disabled.

TASK 6 – READ AND PRINT DATA CORRECTLY

1. Aim

To use `scanf()` and `printf()` with correct format specifiers, validate user input, and display electricity bills accurately in rupees and paise.

2. Algorithm

1. Start.
2. Read the electricity units using `scanf()`.
3. Check the return value of `scanf()` to detect invalid input.
4. Calculate the bill in integer paise.
5. Convert the total paise into rupees and remaining paise.
6. Display the bill using the correct format specifiers.
7. Stop.

3. C Program

```c
#include <stdio.h>
int main(void)
{
    int units;
    long bill_paise;
    printf("Enter electricity units: ");
    if (scanf("%d", &units) != 1 || units < 0)
    {
        printf("Invalid input!\n");
        return 1;
    }
    bill_paise = 5000L;
    if (units > 200)
        bill_paise += (long)(units - 200) * 450;
    if (units > 100)
    {
        int slab2 = units < 200 ? units - 100 : 100;
        bill_paise += (long)slab2 * 225;
    }
    printf("Units consumed: %d\n", units);
```

```
21      printf("Bill amount: Rs %ld.%02ld\n",
22             bill_paise / 100, bill_paise % 100);
23      return 0;
24  }
```

4. Input and Output

Output

```
Enter electricity units: 150
Units consumed: 150
Bill amount: Rs 162.50
```

5. Format Specifies

Data type	Format specifier
`int`	`%d`
`unsigned int`	`%u`
`long`	`%ld`
`long long`	`%lld`
`float`	`%f` in `printf()` and `%f` in `scanf()`
`double`	`%f` in `printf()` and `%lf` in `scanf()`
`char`	`%c`
String (`char` array)	`%s`

6. Wrong Format Specifier

Using `%d` to print a `long` value is incorrect. For example:

```
long amount = 123456789L;
printf("%d\n", amount);  /* Incorrect */
printf("%ld\n", amount); /* Correct */
```

The incorrect format causes undefined behaviour, so the displayed result may be wrong. The correct specifier for `long` is `%ld`.

7. Alternative Concept and Complexity Analysis

An alternative is to read a complete input line using `fgets()` and then convert it using `sscanf()` or `strtol()`.

- `scanf()` reads input directly and checks whether the expected value was converted successfully.
- `fgets()` reads a line into a fixed-size buffer, and `strtol()` can validate the converted number and detect extra characters or out-of-range values.
- Processing a line of length (L) takes O(L) time. The buffer requires O(L) space, or O(B) space for a fixed buffer of size B.
- `scanf()` can leave invalid characters in the input stream, while line-based input makes it easier to discard an invalid line and request new input.

8. Result

The program successfully reads electricity units, validates the input, calculates the bill in integer paise, and prints the amount correctly in rupees and paise using appropriate format specifies.

TASK 7 – CONTROL TYPE CONVERSION AND PRECISION

1. Aim

To understand type conversion, integer division, explicit casting, and rounding errors in C, and to ensure accurate electricity bill calculations.

2. Algorithm

1. Start.
2. Declare variables of different data types.
3. Evaluate expressions involving implicit conversion, integer division, and explicit casting.
4. Display the results.
5. Calculate money using integer paise and apply a consistent rounding rule.
6. Compare integer and floating-point calculations.
7. Stop.

3. C Program

```c
#include <stdio.h>
int main(void)
{
    int a = 5, b = 2;
    double x = 0.1, y = 0.2;
    long amount = 3000000000L;
    printf("Integer division: %d\n", a / b);
    printf("Floating addition: %.17f\n", x + y);
    printf("Comparison (0.1 + 0.2 == 0.3): %d\n",
           x + y == 0.3);
    printf("Explicit cast: %d\n", (int)3.9);
    printf("Long value: %ld\n", amount);
    printf("Casted int value: %d\n", (int)amount);
    return 0;
}
```

4. Output

Output

```
Integer division: 2
Floating addition: 0.30000000000000004
Comparison (0.1 + 0.2 == 0.3): 0
Explicit cast: 3
Long value: 3000000000
Casted int value: -1294967296
```

5. Explanation of Unexpected Results

1. Integer division

`5 / 2` produces `2`, not `2.5`, because both operands are integers and the fractional part is discarded.

2. Floating-point addition

`0.1 + 0.2` may produce `0.30000000000000004` because decimal fractions cannot always be represented exactly in binary floating-point format.

3. Floating-point comparison

The expression `0.1 + 0.2 == 0.3` produces `0` (false) on a typical IEEE 754 system because the two calculated values are not exactly equal.

4. Explicit casting

`(int)3.9` produces `3` because conversion from floating point to integer discards the fractional part; it does not round to the nearest integer.

5. Large integer conversion

Casting a `long` value that exceeds the range of `int` can change its value. Therefore, a large bill total should not be converted to `int` without checking its range.

6. Type Conversion in Billing Calculations

- **Implicit conversion:** Smaller integer types may be promoted to `int` during arithmetic. When different arithmetic types are combined, the usual arithmetic conversions determine the resulting type.
- **Explicit casting:** `(long)(units - 200) * RATE3_PAISE` converts the first operand to `long` before multiplication, helping prevent overflow in the multiplication when the result fits in `long`.
- **Integer division:** `bill_paise / 100` gives whole rupees, while `bill_paise % 100` gives the remaining paise.
- **Rounding:** If a fractional charge is calculated, convert it to paise using a consistent rule, such as rounding to the nearest paise. For non-negative amounts, this can be expressed as `floor(amount_in_rupees * 100 + 0.5)`, assuming the intermediate value is sufficiently accurate.

7. Alternative Concept and Complexity Analysis

If all bills were calculated using `float`, the total could be calculated as a floating-point sum and rounded to two decimal places at the end. However, floating-point representation errors may accumulate, so rounding only once does not guarantee an exact monetary total.

Feature	Integer paise	Float rupees
Typical size	8 bytes using `long long`	4 bytes
Addition time	O(1)	O(1)
Total time for n bills	O(n)	O(n)
Precision	Exact whole paise within range	Approximate
Rounding error	None during integer addition	May accumulate

For (n) bills, integer addition takes O(n) time and O(1) extra space when maintaining a running total. Floating-point errors depend on the values and order of addition; they do not necessarily grow at a fixed rate.

8. Result

Type conversion, integer division, explicit casting, and floating-point precision were demonstrated using C expressions. Integer paise is recommended for electricity billing because it avoids fractional-paise errors during addition, provided the total remains within the integer type's range.

TASK 8 – TRACE THE COMPILATION PROCESS

1. Aim

To understand the stages of C compilation, identify errors and warnings, and observe how source code is converted into an executable program.

2. Algorithm

1. Start.
2. Write and save the C program as `main.c`.
3. Run the preprocessor using `gcc -E`.
4. Generate assembly code using `gcc -S`.
5. Generate an object file using `gcc -c`.
6. Link the object file to create an executable.
7. Compile with `-Wall -Wextra` to check warnings.
8. Observe the results and stop.

3. Compilation Stages and Commands

Run the following commands in a terminal in the folder containing `main.c`.

Stage 1: Pre-processing

```
gcc -E main.c -o main.i
```

Output: `main.i` contains preprocessed source code. Header contents are expanded, macros are replaced, and conditional compilation directives are processed.

Stage 2: Compilation to Assembly

```
gcc -S main.i -o main.s
```

Output: `main.s` contains assembly language instructions.

Stage 3: Assembly to Object File

```
gcc -c main.s -o main.o
```

Output: `main.o` contains machine code and symbol information but is not yet a complete executable.

Stage 4: Linking

```
gcc main.o -o main
```

Output: `main` is the executable program on Linux.

Run it using:

```
./main
```

Checking warnings:

```
gcc -Wall -Wextra main.c -o main
```

These options enable additional compiler warnings to help identify suspicious code, unused variables, and other potential problems.

4. Types of Errors

Compile-time error: Occurs when the compiler detects invalid syntax or a type-related error. Example: missing a semicolon after a statement.

Link-time error: Occurs when the linker cannot find a required function definition. Example: calling `compute_bill()` without linking the source file that defines it.

Run-time error: Occurs while the program is executing. Example: dividing an integer by zero.

5. Warnings and Their Resolution

Warnings from `gcc -Wall -Wextra` identify code that may be incorrect or unsafe. For example, using `%d` to print a `long` value produces a format warning. It can be resolved by using `%ld`.

Warnings should be checked and corrected where appropriate. A program that compiles successfully may still contain warnings or logical errors.

6. Alternative Concept – Multiple Source Files

If the program is divided into `input.c`, `tariff.c`, and `report.c`, with a shared header file, each source file can be compiled separately:

```
gcc -c input.c -o input.o
gcc -c tariff.c -o tariff.o
gcc -c report.c -o report.o
gcc input.o tariff.o report.o -o billing
```

When only one source file changes, only that file normally needs recompilation, followed by linking. In a single-file program, changing the source requires recompiling the entire file.

Separate compilation can save time in large projects. Each object file occupies disk space, and the total object-file space depends on the generated machine code and symbol information. Header changes may also require recompiling source files that include the header.

7. Complexity Analysis

- Compilation time depends on source-code size and compiler optimisation.
- Linking combines object files and resolves external references.
- The object files require additional disk space before linking.
- At runtime, the billing calculation takes O(1) time per bill, while processing (n) bills takes O(n) time.

8. Result

The C compilation process was studied through preprocessing, compilation, assembly, and linking. Compiler warnings and compile-time, link-time, and run-time errors were identified, and separate compilation was examined as an efficient method for organising larger programs.

TASK 9 – IMPLEMENT AND TEST THE BILLING PROGRAM

1. Aim

To implement and test a C electricity billing program using slab rates, validate meter readings, handle meter rollover, and verify the correctness of bill calculations.

2. Algorithm

1. Start.

2. Read the previous and current meter readings.
3. Validate the inputs and ensure both readings are between 0 and 65,535.
4. If the current reading is lower, check whether a valid rollover has occurred.
5. Calculate the units consumed.
6. Calculate the bill using the fixed charge and slab rates.
7. Display the units consumed and the bill amount.
8. Stop.

3. Complete C Program

main.c Run ▶

```c
#include <stdio.h>
#define METER_MAX 65535
#define FIXED_CHARGE_PAISE 5000L
#define RATE2_PAISE 225
#define RATE3_PAISE 450
long compute_bill(int units)
{
    long paise = FIXED_CHARGE_PAISE;

    if (units > 200)
        paise += (long)(units - 200) * RATE3_PAISE;

    if (units > 100) {
        int slab2 = units < 200 ? units - 100 : 100;
        paise += (long)slab2 * RATE2_PAISE;
    }
    return paise;
}
int main(void)
{
    int previous, current, units;
    long bill;
    printf("Enter previous meter reading: ");
    if (scanf("%d", &previous) != 1) {
        printf("Invalid input!\n");
        return 1;
    }
    printf("Enter current meter reading: ");
    if (scanf("%d", &current) != 1) {
        printf("Invalid input!\n");
        return 1;
    }
    if (previous < 0 || previous > METER_MAX ||
        current < 0 || current > METER_MAX) {
        printf("Invalid meter reading!\n");
        return 1;
    }
    if (current < previous) {…} else {
        units = current - previous;
```

```
    }
    bill = compute_bill(units);

    printf("Units consumed: %d\n", units);
    printf("Bill amount: Rs %ld.%02ld\n",
           bill / 100, bill % 100);
    return 0;
}
```

Output

```
Enter previous meter reading: 1000
Enter current meter reading: 1150
Units consumed: 150
Bill amount: Rs 162.50
```

4. Test Cases and Results

The following expected results use a fixed charge of ₹50, the first 100 units free, the next 100 units at ₹2.25 per unit, and units above 200 at ₹4.50 per unit.

Test	Input (previous, current)	Expected result
Zero units	1000, 1000	₹50.00
Exactly 100 units	1000, 1100	₹50.00
150 units	1000, 1150	₹162.50
Exactly 200 units	1000, 1200	₹275.00
350 units	1000, 1350	₹950.00
Meter rollover	65400, 120	256 units; ₹302.00
Invalid letters	`abc`, any value	Invalid input
Negative reading	-10, 100	Invalid meter reading
Invalid decrease	1000, 900	Rollover not accepted

Note: The rollover validation uses 65,000 as a practical threshold for this exercise. A real meter system should follow the meter manufacturer's rollover and fault-detection rules.

5. Five-Bill Calculation

Consider five bills for consumption of 0, 100, 150, 200, and 350 units.

Units	Bill amount
0	₹50.00
100	₹50.00
150	₹162.50
200	₹275.00
350	₹950.00
Total	**₹1,487.50**

The total is calculated by adding all five bill amounts manually. The program can be tested with the corresponding meter readings from the previous table.

6. Alternative Concept – Arrays and Loops

Instead of fixed `if` statements, the tariff limits and rates can be stored in arrays. A loop can calculate the charge for each slab.

For example:

```
int limits[] = {100, 200};
int rates[] = {0, 225, 450};
```

The program would iterate through the slabs, calculate the units falling within each slab, and add the corresponding charge.

- **Fixed `if` statements:** A constant number of checks, giving O(1) time per bill for a fixed tariff structure.
- **Array and loop method:** O(s) time per bill, where `s` is the number of slabs.
- **Extra space:** O(s) to store the limits and rates.
- **Maintainability:** Arrays and loops are easier to update when the number of tariff slabs changes.

7. Result

The electricity billing program was implemented using C. Normal slab calculations, boundary values, meter rollover, and invalid inputs were considered. The five-bill total was calculated as ₹1,487.50, and the program's results can be compared with the expected values in the test table.

TASK 10 – Compare Expectations with Execution

1. Aim

To compare the expected and actual behaviour of data types and constants in the electricity-billing program, including floating-point accuracy, integer overflow, storage sizes, and district-level bill totals.

2. Float vs. Integer Paisa Totals

For this example, assume every bill is ₹0.10. The exact total is calculated by multiplying the number of bills by 10 paisa.

Number of bills	Exact total	Float total (approx.)	Difference
4,000	₹400.00	₹400.0152	₹0.0152
60,000	₹6,000.00	₹6,003.5059	₹3.5059
800,000	₹80,000.00	₹80,645.8438	₹645.8438

These are illustrative results from repeatedly adding the single-precision float value 0.10. They are not measurements from my computer.

The float total becomes less accurate as the number of additions increases because many decimal fractions cannot be represented exactly in binary floating-point. Integer paise gives exact totals for these amounts: 4,000 bills = 40,000 paise, 60,000 bills = 600,000 paise, and 800,000 bills = 8,000,000 paise.

3. Integer Overflow at District Scale

Overflow occurs when an integer calculation exceeds the range of its data type.

- **int:** May overflow if the total exceeds its maximum value. A 16-bit int cannot store 8,000,000 paise, but a typical 32-bit int can.
- **long:** A typical 32-bit or 64-bit long can store 8,000,000 paise.
- **long long:** A typical 64-bit long long can store this total comfortably.

Therefore, the actual result depends on the system's type sizes and the size of each bill. Signed integer overflow in C is undefined behaviour, so it must be prevented.

4. Size of and C Standard Minimums

The C standard does not require every computer to use the same number of bytes for each integer type. It guarantees minimum ranges, while actual sizes depend on the implementation.

Data type	Minimum guaranteed range
int	At least −32,767 to 32,767
long	At least −2,147,483,647 to 2,147,483,647
long long	At least −9,223,372,036,854,775,807 to 9,223,372,036,854,775,807

The size of operator and the limits.h header can be used to check the actual sizes and ranges on my system. These results should be compared with the minimum guarantees of the C standard.

Coding

main.c Run ▶

```c
#include <stdio.h>
#include <limits.h>
int main(void)
{
    printf("sizeof(int) = %zu bytes\n", sizeof(int));
    printf("sizeof(long) = %zu bytes\n", sizeof(long));
    printf("sizeof(long long) = %zu bytes\n", sizeof(long
long));
    printf("sizeof(float) = %zu bytes\n", sizeof(float));
    printf("sizeof(double) = %zu bytes\n", sizeof(double));
    printf("INT_MAX = %d\n", INT_MAX);
    printf("LONG_MAX = %ld\n", LONG_MAX);
    printf("LLONG_MAX = %lld\n", LLONG_MAX);
    return 0;
}
```

Output

Output

```
sizeof(int) = 4 bytes
sizeof(long) = 8 bytes
sizeof(long long) = 8 bytes
sizeof(float) = 4 bytes
sizeof(double) = 8 bytes
INT_MAX = 2147483647
LONG_MAX = 9223372036854775807
LLONG_MAX = 9223372036854775807
```

5. Alternative Concept: Using double

Using double instead of long long avoids integer overflow for many large totals, but it does not guarantee exact representation of decimal fractions such as ₹0.10. Repeated additions can introduce small rounding errors.

For 800,000 bills of ₹0.10, double precision typically gives a result extremely close to ₹80,000.00, while integer paise give the exact total of 8,000,000 paisa, provided the integer type has sufficient range.

For fixed-size types, each addition takes constant time, O(1), and each variable uses constant space, O(1). A double often occupies 8 bytes and a long long often occupies 8 bytes on modern systems, but actual sizes must be checked using size of.

6. Conclusion / Result

The comparison shows that integer paise is suitable for accurate monetary calculations, while floating-point totals may accumulate rounding errors. The choice between int, long, and long long must depend on the required range and the system's actual type sizes. Using sizeof, limits.h, and measured program outputs helps verify that the billing program remains reliable when processing large numbers of households.

TASK 11 – Recommend and Defend the Final Design

1. Aim

To recommend suitable data types, constants, and preprocessor features for the electricity-billing program based on accuracy, storage capacity, maintainability, and experimental results.

2. Recommended Data Types

- **unsigned short:** For meter readings when the meter register is confirmed to use the range 0–65535.

- **int:** For household numbers, categories, and individual consumption values when their ranges are sufficient.
- **long long:** For storing accumulated monetary totals in integer paise, because it provides a large range and avoids floating-point rounding errors.
- **float and double:** For measurements requiring fractions, but not as the primary representation for money when exact paisa accuracy is required.

The data types must be checked using `sizeof()` and the limits provided in `limits.h` and `float.h`.

3. Recommended Constants and Pre-processor Features

Tariff slab limits, energy rates, and fixed charges should be defined using named constants.

For example:

```
#define SLAB1_UNITS 100
#define SLAB2_UNITS 200
#define RATE1_PAISE 0
#define RATE2_PAISE 225
#define RATE3_PAISE 450
#define FIXED_CHARGE_PAISE 5000LL
```

These constants make the program readable and reduce errors caused by repeating tariff values throughout the code.

4. Tariff Revision

When the tariff changes, the administrator can update the corresponding named constants and recompile the program. This approach is simple and suitable when tariff revisions are infrequent.

However, the program must be tested again after every revision to ensure that the new rates and slab limits produce correct bills.

5. Limitations of the Design

1. Compiled-in tariff values require recompilation whenever rates change.
2. The meter rollover rule must match the actual meter behaviour.
3. Integer totals must be checked to prevent overflow.
4. Incorrect input data may produce incorrect bills unless validation is performed.
5. The design assumes that the supplied tariff rules and household categories are correct.

6. Correctness at District Scale

The design can remain correct at district scale if it uses appropriate data types, validates inputs, handles meter rollover correctly, and checks the limits of accumulated totals.

Integer paise provide exact monetary arithmetic as long as the values remain within the chosen integer type's range. A `long long` total is suitable for large accumulations, but overflow must still be prevented.

7. Alternative Concept: Configuration File

Instead of compiling tariff values into the program, the co-operative could store them in a configuration file and load them when the program starts.

Reading and validating *t* tariff values takes approximately **O(t) time**, and storing them requires **O(t) space**. For a small number of tariff values, the additional start-up cost is negligible.

The main advantage is that tariff revisions can be made by updating the configuration file without recompiling and redistributing the program. However, the program must validate the file and reject missing, invalid, or inconsistent values.

8. Experimental Evidence

The earlier data-type and total calculations support the recommended design. The float-versus-paise experiment demonstrates the risk of floating-point rounding, while the `sizeof()` and range tests identify whether the selected integer types can hold the required values.

Actual results to record:

- `sizeof(int)`: _____ bytes
- `sizeof(long)`: _____ bytes
- `sizeof(long long)`: _____ bytes
- Exact paisa total for 800,000 bills: _____ paise
- Float total for the same test: _____
- Observed difference: _____

These measurements should be taken from the actual program output.

9. Conclusion / Result

The recommended design uses suitable integer types, integer paise for monetary calculations, named tariff constants, and input validation. It provides accurate and maintainable billing when the chosen types have sufficient range. For frequent tariff revisions, a validated configuration file is more flexible than compiled-in constants.

TASK 12 – Re-evaluate and Adapt the Design

A. Impact Analysis

1. Changes from the Original Problem

The original program calculated bills using previous and current meter readings and fixed tariff constants. The new requirements introduce smart-meter readings every 15 minutes, a time-of-day tariff, and tariff updates without recompilation.

Each household generates: 35,040 readings per year = 365 × 24 × 4.

Electricity consumed between 6 PM and 10 PM costs 20% more than the normal rate.

2. Data Types and Constants Affected

- `unsigned short` is insufficient for yearly energy totals because the accumulated value may exceed 65,535.
- `int` may be insufficient for large yearly totals.
- `long long` is suitable for larger energy totals and monetary calculations, provided overflow is checked.
- Tariff rates, fixed charges, and peak-hour multipliers should be loaded from a configuration file instead of being permanently defined using `#define`.

B. Evaluation of the Original Design

Assume each 15-minute interval records between 0 and 65,535 Wh. This is an illustrative upper-bound assumption, not a limit specified for the new smart meter.

Maximum yearly energy: 35,040 × 65,535 = 2,296,346,400 Wh.

This exceeds the maximum signed 32-bit integer value of 2,147,483,647. Therefore, a typical 32-bit signed `int` cannot safely store this yearly total.

The yearly total also exceeds the maximum value of a 16-bit unsigned integer, 65,535. A `long long` is more suitable because its minimum guaranteed range is much larger. The original `#define` tariff constants are also unsuitable for the new requirement because changing them requires recompilation. Runtime configuration is more flexible.

C. Adaptation or Redesign

The revised design uses `long long` for accumulated watt-hours and monetary calculations in integer paise. Tariff values are read from a configuration file. The program applies a 20% increase to energy consumed during peak hours (6 PM to 10 PM).

Configuration file: `tariff.cfg`

```
450 5000 120
```

The values represent:

- Normal energy rate: 450 paise per kWh.

- Fixed charge: 5,000 paise.
- Peak-hour multiplier: 120%.

Revised C Program

main.c Run

```c
#include <stdio.h>
int main(void)
{
    long long rate = 450;
    long long fixed_charge = 5000;
    long long total_wh = 0;
    long long charge = 0;
    long long wh, total_bill;
    int hour, i, n = 4;
    for (i = 0; i < n; i++) {
        printf("Enter hour (0-23) and Wh: ");
        if (scanf("%d %lld", &hour, &wh) != 2 ||
            hour < 0 || hour > 23 || wh < 0) {
            printf("Invalid input!\n");
            return 1;
        }
        total_wh += wh;
        if (hour >= 18 && hour < 22)
            charge += wh * rate * 120;
        else
            charge += wh * rate * 100;
    }
    charge = (charge + 50000) / 100000;
    total_bill = charge + fixed_charge;
    printf("\nTotal consumption: %lld Wh\n", total_wh);
    printf("Energy charge: Rs. %lld.%02lld\n",
           charge / 100, charge % 100);
    printf("Fixed charge: Rs. %lld.%02lld\n",
           fixed_charge / 100, fixed_charge % 100);
    printf("Total bill: Rs. %lld.%02lld\n",
           total_bill / 100, total_bill % 100);
    return 0;
}
```

Output

```
Output

Enter hour (0-23) and Wh: 17 100
Enter hour (0-23) and Wh: 18 200
Enter hour (0-23) and Wh: 21 100
Enter hour (0-23) and Wh: 22 100

Total consumption: 500 Wh
Energy charge: Rs. 2.52
Fixed charge: Rs. 50.00
Total bill: Rs. 52.52
```

The file `readings.txt` must contain 35,040 lines, each containing the interval's hour (0–23) and energy consumed in watt-hours. The code assumes each record represents interval consumption, not a cumulative meter reading.

D. Evidence-Based Defence

The redesigned program is more suitable because it supports frequent readings, peak-hour pricing, and tariff changes without recompilation.Under the stated illustrative limit, the maximum yearly consumption is 2,296,346,400 Wh, which exceeds a signed 32-bit integer's range. A `long long` can store this amount comfortably.The program stores the accumulated charge in an integer representation and rounds only when converting the final charge to paise. This avoids repeated rounding at every interval. The tariff configuration can be updated without changing the C source code.However, the design still requires input validation, protection against arithmetic overflow, and testing with realistic maximum consumption. Its time complexity is **O(n)** for *n* readings, and its working storage is **O(1)** because readings are processed one at a time.

Conclusion / Result

The original design should be modified rather than retained unchanged. Wider integer types, runtime tariff configuration, and peak-hour calculations make the program more adaptable to smart meters. The revised design is suitable for district-scale processing when the input limits and accumulated arithmetic are validated.

CONCLUSION

This assignment focused on designing, implementing, testing, and improving a reliable electricity billing system for rural electricity co-operatives using the C programming language. Through the twelve tasks, the important concepts of problem analysis, data type selection, constants, macros, header files, input/output operations, type conversion, compilation, program testing, and design evaluation were studied and applied to the billing problem.

The billing system was designed to calculate electricity consumption from meter readings, apply the specified tariff slabs, add the fixed charge, and display the final bill accurately. Input validation and appropriate handling of meter rollover help prevent incorrect calculations. The use of suitable integer data types and integer paise for monetary values improves calculation accuracy,

while named constants and organised program files make the code easier to understand, maintain, and update.

The testing and comparison tasks highlighted the importance of checking boundary values, invalid inputs, data type limits, floating-point precision, and integer overflow. The final task extended the original design to support smart-meter readings at 15-minute intervals, peak-hour pricing, and tariff updates through a configuration file. These changes make the system more adaptable to changing electricity billing requirements.

In conclusion, a reliable billing program requires more than correct arithmetic. It must combine accurate calculations, proper input validation, suitable data types, systematic testing, and maintainable program design. With realistic tariff rules, verified meter behaviour, and adequate overflow protection, the proposed C-based system can serve as a foundation for accurate and efficient electricity billing at household and district levels.

REFERENCES

1. Brian W. Kernighan and Dennis M. Ritchie, *The C Programming Language*, 2nd Edition, Prentice Hall, 1988.
2. International Organization for Standardization (ISO), *ISO/IEC 9899:2018 — Programming Languages — C*. Available at: https://www.iso.org/standard/74528.html
3. GNU Project, *The GNU C Reference Manual*. Available at: https://www.gnu.org/software/gnu-c-manual/
4. GNU Project, *GCC: Warning Options*. Available at: https://gcc.gnu.org/onlinedocs/gcc/Warning-Options.html

5. SEI CERT C Coding Standard, *INT08-C: Verify That All Integer Values Are in Range*. Available at: https://cmu-sei.github.io/secure-coding-standards/sei-cert-c-coding-standard/recommendations/integers-int/int08-c/
6. cppreference.com, *C Language Reference*. Available at: https://en.cppreference.com/w/c
7. ISO/IEC, *Programming Languages — C: Standard Library Input/Output Functions*. Refer to the C standard's documentation for functions such as scanf(), printf(), and related input/output operations.